

A Voyage to the Kernel



Part 24

Segment: 4.4, Day 23

In the last issue, we looked at how to track the changes in kernel code using GIT. And we have already discussed how to write very simple device drivers. Now, we can start experimenting with code and contribute. If the code is good enough, it will appear in the main kernel tree!

One of the main requests from novices, regarding my column, is to start a new segment on C/C++ programming. So I think it is better to clarify my stand here. Well, I don't think it's a good idea to start another segment to address such matters, mainly due to two reasons. First, it is too late to start any such discussion—we are already meddling with kernel topics. Second, we will be deviating too much from the actual subject.

In order to come out with really novel and sound ideas, you must be good at writing algorithms. This was the reason I dedicated a couple of columns to this aspect earlier. From this point of view, it is important that you know C and C++, in depth. Considering these factors, the best thing I could do is to put up more tutorials online. I have already posted some introductory material that you can find at <http://tecblog.aasisvinayak.com/category/cc>. If you could specify what topics you would like me to cover, I can make posts related to that space. So now let's go back to our core topic.

Miscellaneous character drivers

Though we did discuss character drivers, we didn't go too deep into 'Miscellaneous Character Drivers'. Consider this scenario: You are writing a small device driver, which is nothing but a small add-on or hack to an existing hardware or piece of software. Though you can register this as a new module and load the same (refer to the previous columns for more information), it is not really the recommended way of performing this task.

To do this, the Linux kernel offers an 'interface' that allows developers to register their small drivers. This is the whole purpose of the *misc* driver.

In the previous columns, we have seen the use of major and minor numbers. And we know that if we use a major number to indicate a particular device, then the same driver is used for accessing it, irrespective of the minor number details. As a consequence, you need to register two different major numbers, say DD1 and DD2, to access the same device. Only then can we use it as a 'pointer'. But if you do this in each and every case, then you will be essentially wasting RAM resources.

This can be solved to an extent by using the *misc* driver. You may note that the *misc* driver is assigned with a static major number 10. You can register your mini drivers (or modules) with this driver and assign a number for it.

For your reference, here are mini drivers from the source:

```
#define PS_MOUSE_MINOR      1
#define MS_BUSHOUSE_MINOR  2
#define ATIXL_BUSHOUSE_MINOR 3
/*#define AMIGAMOUSE_MINOR  4  FIXME OBSOLETE */
#define ATARI_MOUSE_MINORS
#define SUN_MOUSE_MINOR     6
#define APOLLO_MOUSE_MINOR  7
#define PC110PAD_MINOR      9
/*#define ADB_MOUSE_MINOR   10  FIXME OBSOLETE */
#define WATCHDOG_MINOR     130 /* Watchdog timer */
#define TEMP_MINOR          131 /* Temperature Sensor */
#define RTC_MINOR           135
#define EFI_RTC_MINOR       136 /* EFI Time services */
#define SUN_OPENPROM_MINOR  139
#define OMAP1_MINOR         140 /* OMAP1 */
#define NVRAM_MINOR         144
#define SGI_MMTIMER         153
#define STORE_QUEUE_MINOR   155
#define I2O_MINOR           166
#define MICROCODE_MINOR     184
#define TUN_MINOR           280
#define MAAVE_MINOR         219 /* ACP/Mave Modem */
#define HPT_MINOR           220
```